

A

SIMPLE PROJECT GUIDE

How the ADAP project works

A beginner-friendly explanation of how synthetic CSV records become searchable timelines, maps, analytics, and reports.

What is ADAP?

ADAP is a demonstration web application. It combines activity records from four synthetic data files and shows them in one simple interface.

Foodpanda

Daraz

Pathao

Uber

Important: all included records are made-up test data. The project does not connect to real provider accounts or live devices.

What a user can do

1

Search

Find a synthetic user using an exact 11-digit phone number.

2

Review

See one combined activity timeline from all four sources.

3

Understand

Use maps, charts, summaries, filters, and a printable report.

The project at a glance

100

shared synthetic users

5,708

included activity records

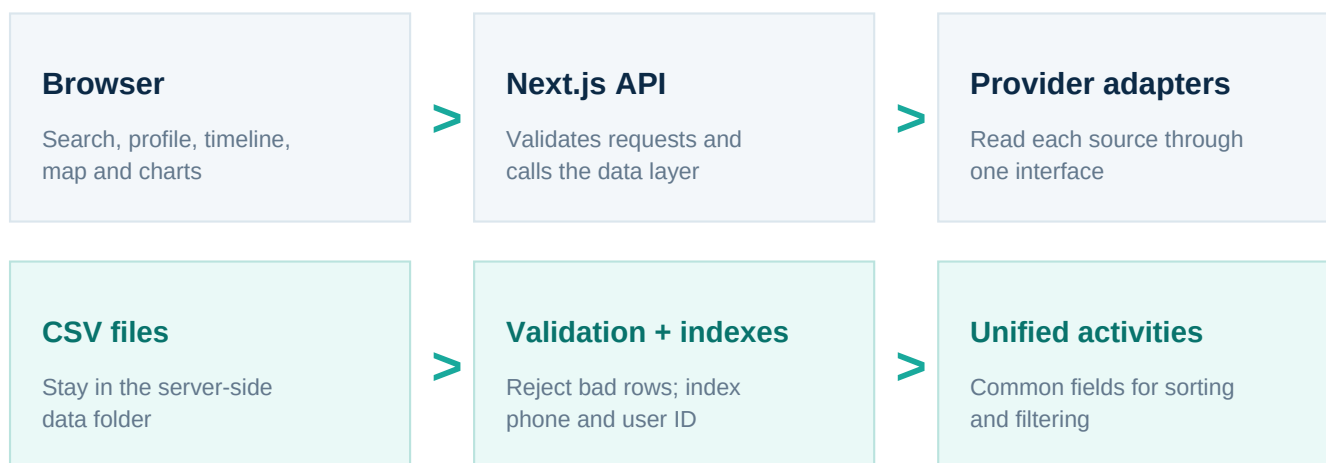
6 months

of demonstration data

Safety boundary: ADAP does not perform live tracking, facial recognition, criminal prediction, guilt scoring, or autonomous targeting.

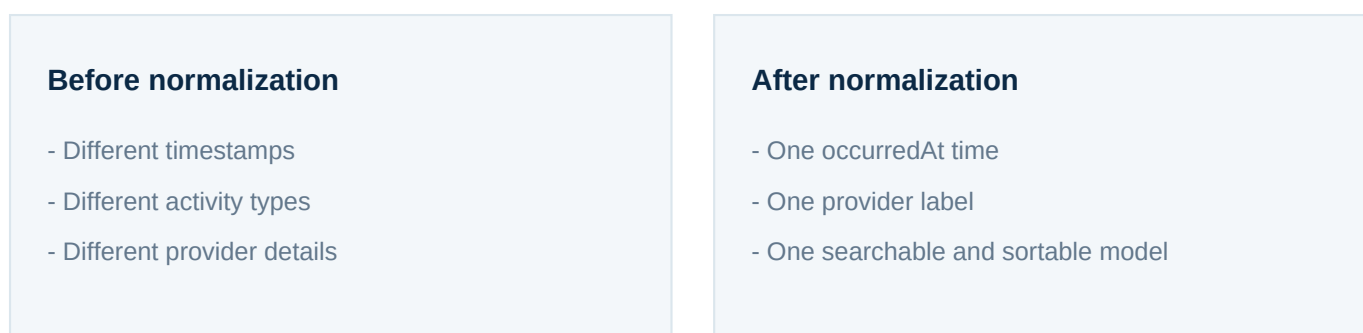
How data moves through the app

The browser never reads the CSV files directly. The server loads, checks, and prepares the data before sending only the requested result to the screen.



Why there are adapters

Each provider CSV has different field names. For example, one source may use an order time while another uses a request time. Each adapter translates its own rows into the same normalized activity shape.



Result: timeline, analytics, map, and report code can work with one model instead of understanding four CSV formats.

What happens when you search

1

Enter a phone number

The user enters an exact 11-digit number, such as 01000000001.

2

Validate the request

The API checks the format. Bad input receives a safe error.

3

Query all adapters

The server asks all four provider adapters at the same time.

4

Use fast indexes

Prepared lookups find matching rows by phone or user ID.

5

Combine the records

Rows are normalized, merged, and sorted by event time.

6

Return only the result

The API sends consistent JSON and the page renders it.

Partial failures do not break everything

If one provider fails, successful results from the others can still be shown. The failed provider is named in response metadata.

Search > validate > query 4 sources > normalize > sort > response > screen

Simple authentication: protected screens use a demonstration session cookie. Replace this with production-grade identity and authorization for real use.

What each screen does

SCREEN	SIMPLE PURPOSE
Dashboard	Search and see provider health, users, and record totals.
Users	Browse a summary-only directory.
Profile	See identity summary, provider sections, notes, and recent activity.
Timeline	Combine and filter activities; export selected data as CSV.
Map	Plot historical points and optionally connect them in event order.
Analytics	Show descriptive charts, date patterns, and provider counts.
Report	Build a printable summary that a browser can save as PDF.
Geofences	Simulate whether historical points entered or exited a circle.
Cases / Audit	Keep demonstration case and UI-action data in this browser.
Data sources	Show adapter status, record counts, and parse health.

Where work happens

On the server

- Read and validate CSV files
- Search and filter activity
- Normalize provider records
- Calculate API results

In the browser

- Display pages and charts
- Interact with maps
- Store local demo items
- Print or Save as PDF

How the code is organized

FOLDER OR FILE	RESPONSIBILITY
<code>data/*.csv</code>	Four synthetic datasets; not in the public web folder.
<code>src/lib/data/</code>	Schemas, parsing, validation, indexes, and normalization.
<code>src/lib/providers/</code>	Common adapter interface and four implementations.
<code>src/app/api/</code>	Validated route handlers with safe JSON responses.
<code>src/app/</code>	Next.js pages and routes.
<code>src/components/</code>	Reusable UI for maps, charts, profiles, cases, and more.
<code>proxy.ts</code>	Demonstration session-cookie gate.

Core technology

Next.js + React Pages, APIs, and interactive UI	Zod Request and CSV validation	csv-parse CSV text to records
Leaflet Historical map points	Recharts Analytics charts	Vitest Automated project tests

Caching in simple words

The first request in a warm server process reads and validates the files. ADAP keeps prepared datasets in memory so later requests can reuse them. Each serverless instance has its own cache.

How to run the project

Requirements: Node.js 20.9 or newer and npm.

```
npm install
npm run dev
```

Open <http://localhost:3000> and use the demonstration account:

Username
analyst

Password
adap123

Useful checks

```
npm test
npm run lint
npm run build
```

The simplest summary

ADAP reads four private synthetic CSV files, validates them, finds matching user records through adapters, converts them into one common activity format, and shows results through demo pages and APIs.

Before any real-world use

- [x] Replace demo login with real identity, roles, and authorization.
- [x] Move local notes, cases, and audit data to protected storage.
- [x] Add encryption, retention rules, monitoring, and data agreements.
- [x] Keep the safety boundary clear: this is a synthetic demonstration.